

# Playwright Tests Explanation

This document explains the logic inside the `playwright_tests.spec.ts` file, breaking down how the test script works and how it catches the errors present in the provided corrupted test files.

## 1. Setup & File Loading

```
test.beforeAll(() => {  
  rawTranscript = JSON.parse(fs.readFileSync(rawTranscriptPath, 'utf8'));  
  metadata = JSON.parse(fs.readFileSync(metadataPath, 'utf8'));  
  processedTranscript = fs.readFileSync(processedTranscriptPath, 'utf8');  
});
```

**What this does:** Before any tests run, this code block uses Node.js's built-in `fs` (file system) module to read the corrupted `.json` and `.txt` files from your directory. It parses the JSON files into JavaScript objects so we can programmatically inspect the data (like checking `metadata.speakers`).

## 2. Data Integrity Tests (Task 1 & 2)

These tests assert that the data behaves according to logical rules. **Because we are running these tests against intentionally corrupted files, these tests are expected to FAIL.** When they fail, it proves that the testing logic successfully caught the bugs!

### TC-1: Entity Names

Checks if the entity names found in `metadata.parties` (e.g., "Cannon Farms") exist perfectly in the processed transcript.

- **Why it fails:** The processed text has AI misspellings like "Cancun Farms" instead of "Cannon Farms".

### TC-2: Speaker Role Consistency

Iterates through all speakers in the metadata and checks if `first_name` + `last_name` equals `full_name`.

- **Why it fails:** Speaker A is listed with first name "Terry", but full name "Jerry Sellingman".

### TC-3, TC-4, TC-5: Timestamp Integrity

These tests loop through every single word in the raw transcript to verify time logic:

- **Monotonicity (TC-3):** Ensures time always moves forward. (Fails because some words go backward in time).
- **Containment (TC-4):** Ensures a word doesn't start *before* the utterance it belongs to.
- **No Duplicates (TC-5):** Ensures two distinct words don't share the exact same starting millisecond.

### TC-6: Valid ISO Date

Takes the `created_at` field from metadata and attempts to parse it into a real JavaScript `Date` object.

- **Why it fails:** The date is `"2025-10-09T26:61:00Z"`. There is no 26th hour or 61st minute, making it an "Invalid Date".

### 3. API & Workflow Testing (Tasks 3 & 4)

Playwright isn't just for clicking around websites in a browser; it has a powerful backend testing tool called `APIRequestContext` (accessible via the `request` fixture).

```
test('Should return 409 Conflict for invalid skips', async ({ request }) => {  
  // const response = await request.post(`${BASE_URL}/workflow/transition`, ...  
});
```

**What this does:** We simulate hitting the endpoints you defined in your `QA_Assessment_Response.md` ( `POST /transcripts/process` and `/workflow/transition` ).

- We check that sending valid data returns a `200 OK` status.
- We test **Edge Cases**, like trying to skip a workflow state (e.g., going straight from `NEW` to `COMPLETED` ), asserting that the API correctly rejects the request by returning a `409 Conflict` status.

*(Note: These API calls are mocked/commented out because we don't have an actual backend server running, but they demonstrate exactly how a Software Engineer in Test (SDET) would automate this!)*